



# GUÍA DE ESTUDIO DEL PROYECTO

## Waldonald's POS

Código, arquitectura, lógica, consultas y base de datos  
Preparación práctica para exposición y preguntas técnicas

|                 |                                                                      |
|-----------------|----------------------------------------------------------------------|
| Objetivo        | Entender y defender el proyecto, no memorizar cada línea.            |
| Enfoque         | Arquitectura, lógica, SQL, base de datos y código crítico.           |
| Cómo estudiarlo | Leer 1-21, practicar 22-24 y responder 25-27 sin ver las respuestas. |

# 1. Qué deben saber del proyecto

La respuesta de 30 segundos y el mapa de estudio.

Waldonald's es un sistema POS de escritorio para restaurante. Permite autenticar usuarios, mostrar productos según horario, configurar productos individuales, WIMenús, Cajitas y combos, personalizar cada componente, cobrar, descontar inventario y consultar reportes administrativos.

La idea técnica principal es que un combo no es un producto plano: es una presentación formada por grupos, opciones y productos internos. Cada producto interno conserva sus propias modificaciones.

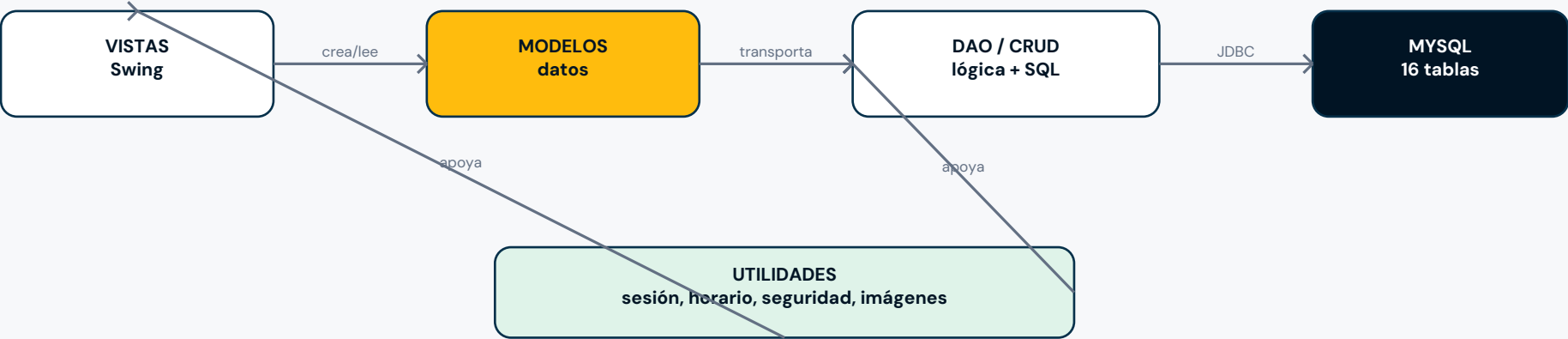
| Tecnología           | Uso en el proyecto                                             | Archivo o carpeta              |
|----------------------|----------------------------------------------------------------|--------------------------------|
| Java 21              | Lenguaje y records para modelos inmutables.                    | src/                           |
| Swing                | Ventanas, paneles, botones, tablas y eventos.                  | GUI_CAJERO / GUI_ADMINISTRADOR |
| NetBeans GUI Builder | Diseño editable de formularios; genera initComponents.         | archivos .form                 |
| JDBC                 | Conexión y consultas parametrizadas a MySQL.                   | Conexion, DAO, CRUD            |
| MySQL                | Persistencia de catálogo, configuración, pedidos e inventario. | sql/                           |
| Ant                  | Compilación y ejecución del proyecto NetBeans.                 | build.xml / nbproject          |

- Roles: ADMINISTRADOR y CAJERO.
- Clase principal: Main.main.
- Base de datos: waldonalds, con 16 tablas.
- Patrón general: Vista → Modelo → DAO/CRUD → MySQL.

## 2. Arquitectura y responsabilidades

Quién hace qué y por qué no debe mezclarse todo en una ventana.

### Capas del sistema



| Carpeta           | Responsabilidad                                         | Regla para explicarla                                |
|-------------------|---------------------------------------------------------|------------------------------------------------------|
| GUI_CAJERO        | Interacción de venta y carrito.                         | No decide por sí sola si el pago es válido.          |
| GUI_ADMINISTRADOR | Mantenimiento de usuarios, menú, inventario y reportes. | Coordina formularios y refresca datos.               |
| Modelos           | Representar productos, opciones, pedido y resultados.   | No contiene SQL ni dibuja pantallas.                 |
| DAO               | Casos de uso y consultas complejas.                     | Devuelve modelos, maneja conexiones y transacciones. |
| CRUD              | Altas, lecturas, cambios y estado de catálogos.         | Centraliza validaciones administrativas.             |
| Componentes       | Estilo visual reutilizable.                             | Evita duplicar el mismo botón o tabla.               |
| Utilidades        | Funciones compartidas.                                  | Sesión, horarios, hash, CSV, iconos e imágenes.      |

### 3. Arranque de la aplicación

Main main. EDT y transición de la pantalla de carga al Login

```
7 | public static void main(String[] args) {
8 |     java.awt.EventQueue.invokeLater(new Runnable() {
9 |         @Override
10 |         public void run() {
11 |
12 |             PantallaCarga carga = new PantallaCarga();
13 |             carga.setLocationRelativeTo(null);
14 |             carga.setVisible(true);
15 |
16 |             // Esperar 3 segundos
17 |             Timer timer = new Timer(8000, e -> {
18 |
19 |                 // Cerrar pantalla de carga
20 |                 carga.dispose();
21 |
22 |                 // Abrir siguiente interfaz
23 |                 Login login = new Login();
24 |                 login.setLocationRelativeTo(null);
25 |                 login.setVisible(true);
26 |             });
27 |
28 |             timer.setRepeats(false);
29 |             timer.start();

```

| Línea o concepto | Qué significa y por qué importa                                                                                                                              |
|------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 7                | main es el punto de entrada que ejecuta la JVM.                                                                                                              |
| 8                | EventQueue.invokeLater coloca la creación de la interfaz en el hilo visual de Swing (EDT).                                                                   |
| 12-14            | Crea, centra y muestra PantallaCarga.                                                                                                                        |
| 17               | Timer espera 8000 ms sin bloquear la interfaz. El comentario dice 3 segundos, pero el valor real es 8; en una exposición deben reconocer esa inconsistencia. |
| 20               | dispose libera la ventana de carga.                                                                                                                          |
| 23-25            | Crea, centra y muestra Login.                                                                                                                                |
| 28               | setRepeats(false) impide que el Login se abra repetidamente.                                                                                                 |

Pregunta típica: ¿por qué no usar Thread.sleep? Porque bloquearía el EDT y congelaría la interfaz. Timer programa la acción manteniendo Swing sensible.

## 4. Inicio de sesión, seguridad y roles

```

134 |         String sql = ""
135 |             SELECT id_usuario,
136 |                   nombre,
137 |                   apellido,
138 |                   rol,
139 |                   password_hash
140 |             FROM usuario
141 |            WHERE usuario = ?
142 |              AND estado = TRUE
143 |             "";
144 |
145 |         try (Connection con = conexion; PreparedStatement ps = con.prepareStatement(sql)) {
146 |
147 |             ps.setString(1, usuario);
148 |
149 |             try (ResultSet rs = ps.executeQuery()) {
150 |
151 |                 if (rs.next()) {
152 |                     String hashAlmacenado = rs.getString("password_hash");
153 |                     if (!SeguridadContraseña.verificar(contraseña, hashAlmacenado)) {
154 |
155 |                         ...
156 |
157 |                         int idUsuario = rs.getInt("id_usuario");
158 |                         String nombre = rs.getString("nombre");
159 |                         String apellido = rs.getString("apellido");
160 |                         String rol = rs.getString("rol");
161 |
162 |                         // Guarda la identidad mínima que necesitan las pantallas.
163 |                         SesionUsuario.iniciar(idUsuario, nombre, apellido, rol);
164 |
165 |                         ...
166 |
167 |                         if (rol.equalsIgnoreCase("ADMINISTRADOR")) {
168 |
169 |                             InicioAdminForm ventanaAdministrador
170 |                                 = new InicioAdminForm();
171 |
172 |                             ventanaAdministrador.setLocationRelativeTo(null);
173 |                             ventanaAdministrador.setVisible(true);
174 |
175 |                             dispose();
176 |
177 |                         } else if (rol.equalsIgnoreCase("CAJERO")) {
178 |
179 |                             Cajero ventanaCajero = new Cajero();
180 |
181 |                             ventanaCajero.setLocationRelativeTo(null);
182 |                             ventanaCajero.setVisible(true);
183 |
184 |                             dispose();
185 |
186 |                         }
187 |
188 |                     }
189 |                 }
190 |             }
191 |         }
192 |     }
193 | }

```

| Línea o concepto | Qué significa y por qué importa                                                      |
|------------------|--------------------------------------------------------------------------------------|
| 134-143          | Consulta solo el usuario solicitado y exige estado=TRUE.                             |
| 145              | try-with-resources cierra Connection y PreparedStatement incluso si ocurre un error. |
| 147              | setString sustituye el ? de PreparedStatement; evita concatenar texto del usuario.   |
| 151-153          | Lee la fila y verifica la contraseña contra el hash almacenado.                      |
| 177-183          | Extrae identidad mínima y la guarda en SesionUsuario para otras pantallas.           |
| 190-207          | El rol decide si se abre InicioAdminForm o Cajero; luego se cierra Login.            |

Debilidad a reconocer: la conexión tiene credenciales en el código. Para producción deberían provenir de variables de entorno o configuración local no versionada.

## 5. Java y Swing que deben poder explicar

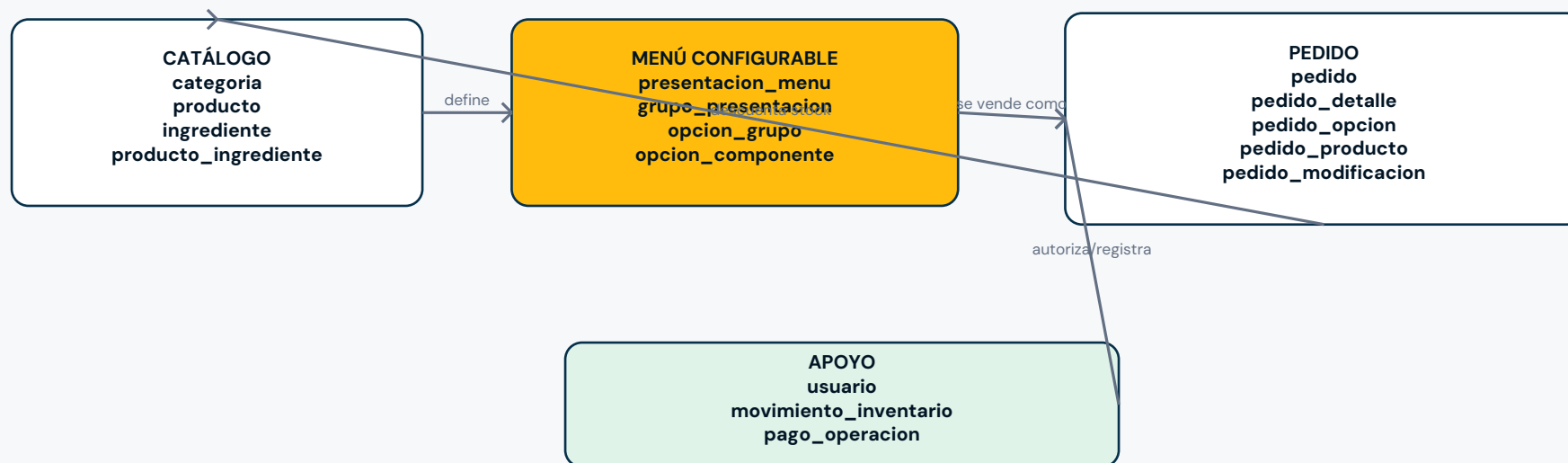
| Concepto           | Significado dentro del proyecto                    | Ejemplo                      |
|--------------------|----------------------------------------------------|------------------------------|
| class              | Molde mutable con atributos y métodos.             | Producto, Usuario            |
| record             | Modelo compacto e inmutable; Java crea accesores.  | LineaPedido, OpcionPedido    |
| constructor        | Valida y prepara un objeto al usar new.            | public LineaPedido {...}     |
| private/public     | Control de acceso dentro o fuera de la clase.      | private void confirmar       |
| final              | La referencia no puede reasignarse.                | private final Map...         |
| listener           | Reacciona a clic, tecla o cambio.                  | addActionListener            |
| CardLayout         | Muestra un panel de varios sin abrir otra ventana. | PedidoPanel, MenuAdmin       |
| SwingWorker        | Ejecuta consultas fuera del EDT para no congelar.  | paneles administrativos      |
| try/catch          | Maneja errores esperados.                          | SQLException                 |
| try-with-resources | Cierra conexión, sentencia y ResultSet.            | try (Connection c=...)       |
| BigDecimal         | Dinero exacto; evita errores de double.            | precio, total, stock decimal |
| List               | Colección ordenada.                                | opciones, productos          |
| Map                | Relaciona una clave con un objeto.                 | idLinea -> LineaPedido       |
| stream             | Procesa colecciones declarativamente.              | map + reduce del total       |

initComponents es código generado a partir del .form. Las marcas //GEN-BEGIN y //GEN-END permiten que NetBeans reconozca el bloque. Si se borran, el diseñador puede quedar en solo lectura.

## 6. Modelo relacional completo

Las 16 tablas se agrupan en catálogo, menú configurable, ventas e inventario.

### Mapa de la base de datos



| Tipo de clave  | Qué significa                                    | Ejemplo                                   |
|----------------|--------------------------------------------------|-------------------------------------------|
| PRIMARY KEY    | Identificador único de cada fila.                | producto.id_producto                      |
| FOREIGN KEY    | Apunta a una fila de otra tabla.                 | producto.id_categoria -> categoria        |
| AUTO_INCREMENT | MySQL genera el siguiente ID.                    | pedido.id_pedido                          |
| UNIQUE         | Impide repetir un valor.                         | usuario.usuario, pago_operacion.id_pedido |
| ENUM           | Limita un campo a opciones conocidas.            | tipo_stock, rol, tipo_servicio            |
| estado         | Borrado lógico; desactiva sin destruir historia. | TRUE/FALSE                                |

## 7. Tablas de catálogo y recetas

| Tabla                | Para qué sirve                                                                                        | Campos que deben reconocer                                                                                                                                             | Relaciones clave                                                                     |
|----------------------|-------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| categoria            | Agrupar productos en las tarjetas principales del menú: Desayuno, Almuerzos, Bebidas, etc.            | id_categoria, nombre, estado                                                                                                                                           | Sin clave foránea                                                                    |
| producto             | Catálogo general. Incluye productos vendibles y productos internos usados como componentes de combos. | id_producto, id_categoria, nombre, descripcion, precio_base, imagen, subcategoria, disponibilidad_menu, tipo_stock, personalizable, stock_actual, stock_minimo, estado | id_categoria -> categoria.id_categoria                                               |
| ingrediente          | Materia prima controlada por inventario, con unidad, existencia y mínimo de reposición.               | id_ingrediente, nombre, unidad_medida, stock_actual, stock_minimo, estado                                                                                              | Sin clave foránea                                                                    |
| producto_ingrediente | Receta de cada producto y reglas de personalización: quitar, extra, cantidad y recargo.               | id_producto_ingrediente, id_producto, id_ingrediente, cantidad_default, permite_quitar, permite_extra, cantidad_extra, precio_extra, max_extras, estado                | id_producto -> producto.id_producto;<br>id_ingrediente -> ingrediente.id_ingrediente |

Ejemplo: Big Mac tiene tipo\_stock=RECETA. Sus ingredientes y cantidades viven en producto\_ingrediente. permite\_quitar y permite\_extra controlan la personalización; precio\_extra y max\_extras controlan el recargo y límite.

## 8. Tablas del menú configurable

| Tabla              | Para qué sirve                                                                                        | Campos que deben reconocer                                                                                | Relaciones clave                                                            |
|--------------------|-------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------|
| presentacion_menu  | Formas de vender un producto principal: Individual, WIMenú, Infantil o Combo, cada una con su precio. | id_presentacion, id_producto_principal, nombre, tipo, precio, predeterminada, estado                      | id_producto_principal -> producto.id_producto                               |
| grupo_presentacion | Pregunta o sección dentro de una presentación, con límites mínimo/máximo y permiso de repetir.        | id_grupo, id_presentacion, nombre, minimo, maximo, permite_repetir, visible, permite_personalizar, estado | id_presentacion -> presentacion_menu.id_presentacion                        |
| opcion_grupo       | Respuesta disponible dentro de un grupo, con recargo y opción predeterminada.                         | id_opcion, id_grupo, nombre, incremento_precio, predeterminada, estado                                    | id_grupo -> grupo_presentacion.id_grupo                                     |
| opcion_componente  | Productos concretos que entrega una opción. Una opción puede contener uno o varios productos.         | id_opcion_componente, id_opcion, id_producto, cantidad                                                    | id_opcion -> opcion_grupo.id_opcion;<br>id_producto -> producto.id_producto |

**Jerarquía:** un producto tiene presentaciones; una presentación tiene grupos; un grupo tiene opciones; una opción entrega uno o más productos. Los campos minimo, maximo y permite\_repetir son reglas, no simples textos.

## 8.1 Menú configurable: consultas y relaciones

Cómo pasa el programa de una tarjeta del cajero a sus grupos, opciones, productos reales e ingredientes.

| Tabla              | Relación                      | Campos decisivos                                               | Ejemplo sencillo                  |
|--------------------|-------------------------------|----------------------------------------------------------------|-----------------------------------|
| presentacion_menu  | producto 1 → N presentaciones | tipo, precio, predeterminada, estado                           | Big Mac: Individual o WIMenú      |
| grupo_presentacion | presentación 1 → N grupos     | minimo, maximo, permite_repetir, visible, permite_personalizar | Elige una bebida                  |
| opcion_grupo       | grupo 1 → N opciones          | incremento_precio, predeterminada, estado                      | McPatatas + Q7.00                 |
| opcion_componente  | opción N ↔ N producto         | id_opcion, id_producto, cantidad                               | McPatatas → producto McPatatas x1 |

El signo ? es un parámetro de PreparedStatement. Java lo sustituye con setInt o setString; no se concatena la entrada del usuario.

```
-- 1. Presentaciones del producto seleccionado
SELECT id_presentacion,nombre,precio,predeterminada
FROM presentacion_menu WHERE id_producto_principal=? AND estado=TRUE
ORDER BY predeterminada DESC,id_presentacion;

-- 2. Preguntas de cada presentación
SELECT id_grupo,nombre,minimo,maximo,permite_repetir,visible,permite_personalizar
FROM grupo_presentacion WHERE id_presentacion=? AND estado=TRUE ORDER BY id_grupo;

-- 3. Respuestas disponibles
SELECT id_opcion,nombre,incremento_precio,predeterminada
FROM opcion_grupo WHERE id_grupo=? AND estado=TRUE
ORDER BY predeterminada DESC,id_opcion;

-- 4. Productos reales que entrega la respuesta
SELECT oc.id_producto,oc.cantidad,p.nombre,p.tipo_stock,p.personalizable
FROM opcion_componente oc JOIN producto p ON p.id_producto=oc.id_producto
WHERE oc.id_opcion=? ORDER BY oc.id_opcion,oc.cantidad;
```

Lectura mental: 'Quiero WIMenú' = presentación; 'elige bebida' = grupo; 'Coca-Cola' = opción; 'Coca-Cola mediana x1' = componente. Si el componente es RECETA, el grupo permite personalizar y el producto es personalizable, se consulta producto\_ingrediente para crear los controles SIN y EXTRA.

## 9. Tablas que conservan el pedido

| Tabla               | Para qué sirve                                                                                  | Campos que deben reconocer                                                                                 | Relaciones clave                                                                                                                  |
|---------------------|-------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| pedido              | Cabecera de la venta: orden, cajero, servicio, estado, pago, total y fecha.                     | id_pedido, numero_orden, id_usuario, tipo_servicio, estado, metodo_pago, monto_recibido, total, fecha_hora | id_usuario -> usuario.id_usuario                                                                                                  |
| pedido_detalle      | Cada línea del carrito, incluyendo la presentación elegida, cantidad y subtotal.                | id_detalle, id_pedido, id_presentacion, nombre, presentacion, cantidad, precio_unitario, subtotal          | id_pedido -> pedido.id_pedido;<br>id_presentacion -> presentacion_menu.id_presentacion                                            |
| pedido_opcion       | Captura las elecciones realizadas en cada grupo de la línea para conservar un historial exacto. | id_pedido_opcion, id_detalle, id_grupo, id_opcion, numero, grupo, opcion, precio_extra                     | id_detalle -> pedido_detalle.id_detalle; id_grupo -> grupo_presentacion.id_grupo;<br>id_opcion -> opcion_grupo.id_opcion          |
| pedido_producto     | Expande cada opción a los productos físicos que cocina o entrega el restaurante.                | id_pedido_producto, id_pedido_opcion, id_producto, nombre, cantidad                                        | id_pedido_opcion -> pedido_opcion.id_pedido_opcion;<br>id_producto -> producto.id_producto                                        |
| pedido_modificacion | Cambios aplicados a cada producto interno: SIN ingrediente o EXTRA ingrediente.                 | id_modificacion, id_pedido_producto, id_producto_ingredient, tipo, ingrediente, cantidad, precio_extra     | id_pedido_producto -> pedido_producto.id_pedido_producto;<br>id_producto_ingredient -> producto_ingredient.id_producto_ingredient |

pedido\_detalle, pedido\_opcion, pedido\_producto y pedido\_modificacion guardan nombres históricos. Es un snapshot: si el producto cambia después, la venta antigua sigue diciendo exactamente qué se cobró y preparó.

## 9.1 Pedido guardado: consultas y relaciones

Cómo se convierte la configuración elegida en una venta permanente y consultable.

| Tabla               | Padre                                 | Qué conserva                                      | Uso principal               |
|---------------------|---------------------------------------|---------------------------------------------------|-----------------------------|
| pedido              | usuario                               | servicio, pago, total, fecha y estado             | Cabecera de la venta        |
| pedido_detalle      | pedido + presentacion_menu            | nombre, presentación, cantidad, precio y subtotal | Líneas y reportes de ventas |
| pedido_opcion       | detalle + grupo + opción              | número, nombres e incremento                      | Elecciones del cliente      |
| pedido_producto     | pedido_opcion + producto              | producto real y cantidad                          | Cocina e inventario         |
| pedido_modificacion | pedido_producto + producto_ingredient | SIN/EXTRA, ingrediente, cantidad y precio         | Preparación individual      |

PagoDAO obtiene la clave AUTO\_INCREMENT después de cada INSERT y la usa como clave foránea del siguiente nivel:

```
INSERT INTO pedido(numero_orden,id_usuario,tipo_servicio,estado,metodo_pago,monto_recibido,total)
VALUES(0,?,?,?, 'PAGADO',?, ?,?);
INSERT INTO pedido_detalle(id_pedido,id_presentacion,nombre,presentacion,cantidad,precio_unitario,subtotal) VALUES(?,?,?, ?, ?, ?,?);
INSERT INTO pedido_opcion(id_detalle,id_grupo,id_opcion,numero,grupo,opcion,precio_extra) VALUES(?,?, ?, ?, ?, ?,?);
INSERT INTO pedido_producto(id_pedido_opcion,id_producto,nombre,cantidad) VALUES(?,?, ?,?);
INSERT INTO pedido_modificacion(id_pedido_producto,id_producto_ingredient,tipo_ingredient,cantidad,precio_extra) VALUES(?,?, ?. ?. ?. ?):

-- Reconstruir un pedido completo
SELECT pe.numero_orden,pd.nombre,pd.presentacion,po.grupo,po.opcion,
       pp.nombre AS producto_interno,pm.tipo,pm.ingrediente,pm.precio_extra
FROM pedido pe JOIN pedido_detalle pd ON pd.id_pedido=pe.id_pedido
LEFT JOIN pedido_opcion po ON po.id_detalle=pd.id_detalle
LEFT JOIN pedido_producto pp ON pp.id_pedido_opcion=po.id_pedido_opcion
LEFT JOIN pedido_modificacion pm ON pm.id_pedido_producto=pp.id_pedido_producto
WHERE pe.id_pedido=?;
```

Los nombres y precios repetidos son copias históricas: renombrar Big Mac mañana no cambia el ticket de ayer. Cada unidad interna obtiene su propio pedido\_producto; por eso dos hamburguesas iguales pueden tener modificaciones distintas. Los reportes suman pedido\_detalle, no los componentes, para no contar un WIMenú como tres ventas.

## 10. Usuarios, inventario e idempotencia

| Tabla                 | Para qué sirve                                                                             | Campos que deben reconocer                                                                                              | Relaciones clave                                                                                                  |
|-----------------------|--------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| usuario               | Personas que pueden iniciar sesión. Define credenciales, rol, estado y horario de trabajo. | id_usuario, nombre, apellido, usuario, correo, password_hash, rol, estado, fecha_creacion, turno, hora_inicio, hora_fin | Sin clave foránea                                                                                                 |
| movimiento_inventario | Bitácora de entradas, salidas y ajustes de productos directos o ingredientes.              | id_movimiento, id_producto, id_ingredient, id_pedido, tipo_movimiento, cantidad, motivo, fecha_hora                     | id_producto -> producto.id_producto;<br>id_ingredient -> ingrediente.id_ingredient; id_pedido -> pedido.id_pedido |
| pago_operacion        | Control de idempotencia del cobro: evita registrar dos veces la misma solicitud de pago.   | clave, solicitud, id_pedido, comprobante, referencia                                                                    | id_pedido -> pedido.id_pedido                                                                                     |

movimiento\_inventario puede apuntar a producto o ingrediente. pago\_operacion usa clave como identificador del intento lógico de cobro para que un reintento no duplique el pedido.

## 11. Cómo se representa Individual, WIMenú, Cajita y combo

| Nivel                | Ejemplo             | Responsabilidad                                               |
|----------------------|---------------------|---------------------------------------------------------------|
| producto             | Big Mac             | Tarjeta que abre el cajero y producto que puede tener receta. |
| presentacion_menu    | Individual / WIMenú | Precio base del formato de venta.                             |
| grupo_presentacion   | Elige complemento   | Exige mínimo y máximo de selecciones.                         |
| opcion_grupo         | WIPatatas +Q5       | Alternativa y recargo.                                        |
| opcion_componente    | 1 WIPatatas         | Producto real que entrega la opción.                          |
| producto_ingrediente | Pepinillo           | Regla para SIN/EXTRA dentro de una hamburguesa.               |

```

LineaPedido
-> OpcionPedido: Producto principal
    -> ProductoPedido: Big Mac
        -> ModificacionPedido: SIN Pepinillo
-> OpcionPedido: Complemento
    -> ProductoPedido: WIPatatas
-> OpcionPedido: Bebida
    -> ProductoPedido: Coca Cola
  
```

Combo con dos hamburguesas: se crean dos ProductoPedido distintos, cada uno con idInterno y lista propia de modificaciones. Por eso una puede ir sin pepinillo y la otra con todo.

No se duplica el producto para crear WIMenú. Se añade otra presentacion\_menu al mismo producto principal. Así la tarjeta puede mostrar pestañas Individual y WIMenú.

## 12. Cómo se carga una configuración de menú

```

56 | List<PresentacionMenu> presentaciones = new ArrayList<>();
57 | String sql = "SELECT id_presentacion,nombre,tipo,precio,predeterminada FROM presentacion_menu "
58 | + "WHERE id_producto_principal=?" + (soloActivos ? " AND estado=TRUE" : "")
59 | + " ORDER BY predeterminada DESC,id_presentacion";
60 | try (PreparedStatement s = c.prepareStatement(sql)) {
61 |     s.setInt(1, idProducto);
62 |     try (ResultSet r = s.executeQuery()) {
63 |         while (r.next()) {
64 |             int idPresentacion = r.getInt("id_presentacion");
65 |             List<GrupoMenu> grupos = cargarGrupos(c, idPresentacion, soloActivos, validarHorario);
66 |             if (validarHorario && (grupos.isEmpty() || grupos.stream()
67 |                 .anyMatch(g -> g.minimo() > 0 && g.opciones().isEmpty()))) continue;
68 |             presentaciones.add(new PresentacionMenu(idPresentacion, idProducto,
69 |                 r.getString("nombre"), r.getString("tipo"),
70 |                 r.getBigDecimal("precio"), r.getBoolean("predeterminada"),
71 |                 grupos));
72 |
73 | ...
74 |
75 | String sql = "SELECT id_grupo,nombre,minimo,maximo,permite_repetir,visible,permite_personalizar "
76 | + "FROM grupo_presentacion WHERE id_presentacion=?"
77 | + (soloActivos ? " AND estado=TRUE" : "") + " ORDER BY id_grupo";
78 | try (PreparedStatement s = c.prepareStatement(sql)) {
79 |     s.setInt(1, idPresentacion);
80 |     try (ResultSet r = s.executeQuery()) {
81 |         while (r.next()) {
82 |             int idGrupo = r.getInt("id_grupo");
83 |             grupos.add(new GrupoMenu(idGrupo, r.getString("nombre"),
84 |                 r.getInt("minimo"), r.getInt("maximo"),
85 |                 r.getBoolean("permite_repetir"), r.getBoolean("visible"),
86 |                 r.getBoolean("permite_personalizar"),
87 |                 cargarOpciones(c, idGrupo, soloActivos, validarHorario)));
88 |
89 | ...
90 |
91 | String sql = "SELECT id_opcion,nombre,incremento_precio,predeterminada FROM opcion_grupo "
92 | + "WHERE id_grupo=?" + (soloActivos ? " AND estado=TRUE" : "")
93 | + " ORDER BY predeterminada DESC,id_opcion";
94 | try (PreparedStatement s = c.prepareStatement(sql)) {
95 |     s.setInt(1, idGrupo);
96 |     try (ResultSet r = s.executeQuery()) {
97 |         while (r.next()) {
98 |             int idOpcion = r.getInt("id_opcion");
99 |             List<ComponenteMenu> componentes = cargarComponentes(c, idOpcion, soloActivos, validarHorario);
100 |             if (soloActivos && componentes.size() != contarComponentes(c, idOpcion)) continue;
101 |             opciones.add(new OpcionMenu(idOpcion, r.getString("nombre"),
102 |                 r.getBigDecimal("incremento_precio"),
103 |                 r.getBoolean("predeterminada"), componentes));
104 |
105 | ...
106 |
107 | String sql = "SELECT id_opcion,nombre,incremento_precio,predeterminada FROM opcion_grupo "
108 | + "WHERE id_grupo=?" + (soloActivos ? " AND estado=TRUE" : "")
109 | + " ORDER BY predeterminada DESC,id_opcion";
110 | try (PreparedStatement s = c.prepareStatement(sql)) {
111 |     s.setInt(1, idGrupo);
112 |     try (ResultSet r = s.executeQuery()) {
113 |         while (r.next()) {
114 |             int idOpcion = r.getInt("id_opcion");
115 |             List<ComponenteMenu> componentes = cargarComponentes(c, idOpcion, soloActivos, validarHorario);
116 |             if (soloActivos && componentes.size() != contarComponentes(c, idOpcion)) continue;
117 |             opciones.add(new OpcionMenu(idOpcion, r.getString("nombre"),
118 |                 r.getBigDecimal("incremento_precio"),
119 |                 r.getBoolean("predeterminada"), componentes));

```

| Línea o concepto | Qué significa y por qué importa                                                          |
|------------------|------------------------------------------------------------------------------------------|
| 56-71            | Consulta presentaciones, prioriza la predeterminada y construye cada una con sus grupos. |
| 66-67            | Si un grupo obligatorio no tiene opciones válidas, la presentación se oculta al cajero.  |
| 84-96            | Lee mínimo, máximo, repetición, visibilidad y personalización; después carga opciones.   |
| 106-118          | Cada opción conserva recargo, selección predeterminada y productos componentes.          |

El DAO convierte filas SQL en un árbol de modelos. La interfaz recibe ConfiguracionProducto ya estructurado y no ejecuta estas consultas.

## 13. Consulta que decide qué productos ve el cajero

```

42 |         SELECT
43 |             id_producto,
44 |             id_categoria,
45 |             nombre,
46 |             descripcion,
47 |             COALESCE((SELECT pm.precio FROM presentacion_menu pm
48 |                 WHERE pm.id_producto_principal=p.id_producto AND pm.estado=TRUE
49 |                 ORDER BY pm.predeterminada DESC,pm.id_presentacion LIMIT 1), precio_base) AS precio_b...
50 |             imagen,
51 |             disponibilidad_menu
52 |         FROM producto p
53 |         WHERE p.id_categoria = ?
54 |         AND p.estado = TRUE
55 |         AND EXISTS (SELECT 1 FROM presentacion_menu pm
56 |             WHERE pm.id_producto_principal=p.id_producto AND pm.estado=TRUE)
57 |         "" : ""
58 |         SELECT id_producto, id_categoria, nombre, descripcion,
59 |             precio_base, imagen, disponibilidad_menu
60 |         FROM producto
61 |         WHERE id_categoria = ?
62 |         AND estado = TRUE
63 |         "" ;

```

| Línea o concepto     | Qué significa y por qué importa                                                     |
|----------------------|-------------------------------------------------------------------------------------|
| COALESCE             | Usa el precio de una presentación activa; si no existe, cae a precio_base.          |
| subconsulta          | Busca una presentación del mismo producto principal.                                |
| WHERE id_categoria=? | Filtra por la categoría elegida; el valor se asigna con PreparedStatement.          |
| estado=TRUE          | Oculto productos desactivados sin borrarlos.                                        |
| EXISTS               | Exige al menos una presentación activa; evita tarjetas que no se pueden configurar. |
| ORDER BY ... LIMIT 1 | Prefiere la presentación predeterminada y devuelve solo una.                        |

Además del SQL, HorarioMenu valida disponibilidad\_menu. TODO\_DIA siempre; DESAYUNO y ALMUERZO dependen de la hora. Por eso un WIMuffin no debe aparecer de noche.

## 14. Modelo del carrito y cálculo del precio

```

7 | /** Una línea del carrito. Dos configuraciones distintas nunca se mezclan. */
8 | public record LineaPedido(String idLinea, int idProducto, int idPresentacion,
9 |     String nombre, String presentacion, BigDecimal precioBase,
10 |     List<OpcionPedido> opciones, int cantidad) {
11 |     public LineaPedido {
12 |         if (idLinea == null || idLinea.isBlank()) idLinea = UUID.randomUUID().toString();
13 |         if (idProducto <= 0 || idPresentacion <= 0 || nombre == null
14 |             || presentacion == null || precioBase == null
15 |             || precioBase.signum() < 0 || precioBase.scale() > 2
16 |             || cantidad < 1 || cantidad > 999) {
17 |             throw new IllegalArgumentException("Producto, presentación, precio o cantidad inválidos.");
18 |         }
19 |         opciones = List.copyOf(opciones == null ? List.of() : opciones);
20 |     }
21 |
22 |     public BigDecimal precio() {
23 |         BigDecimal total = precioBase;
24 |         for (OpcionPedido opcion : opciones) {
25 |             total = total.add(opcion.precioExtra());
26 |             for (ProductoPedido producto : opcion.productos()) {
27 |                 for (ModificacionPedido modificacion : producto.modificaciones())
28 |                     total = total.add(modificacion.precioExtra());
29 |             }
30 |         }
31 |         return total.setScale(2);
32 |     }
33 |
34 |     public BigDecimal subtotal() { return precio().multiply(BigDecimal.valueOf(cantidad)); }
35 |
36 |     public LineaPedido conCantidad(int nuevaCantidad) {
37 |         return new LineaPedido(idLinea, idProducto, idPresentacion, nombre,
38 |             presentacion, precioBase, opciones, nuevaCantidad);

```

| Línea o concepto | Qué significa y por qué importa                                                      |
|------------------|--------------------------------------------------------------------------------------|
| record           | Representa una línea inmutable; dos configuraciones no se mezclan.                   |
| idLinea UUID     | Identifica la configuración exacta dentro del carrito, aunque sea el mismo producto. |
| List.copyOf      | Crea una copia inmutable de opciones para evitar cambios externos.                   |
| precio()         | Comienza con precioBase, suma recargos de opciones y recargos de modificaciones.     |
| setScale(2)      | Mantiene dos decimales monetarios.                                                   |
| subtotal()       | precio configurado multiplicado por cantidad.                                        |
| conCantidad      | Como el record es inmutable, devuelve otra LineaPedido con la nueva cantidad.        |

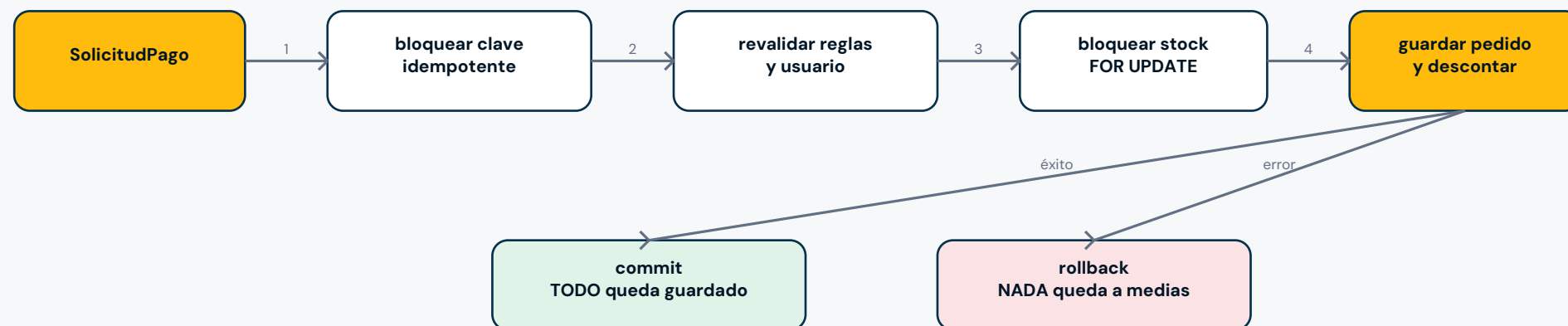
# 15. Del configurador al carrito

```
273 | private void actualizarPrecio() {
274 |     VistaPresentacion vista = vistaActual();
275 |     if (vista == null || agregar == null) {
276 |         return;
277 |     }
278 |     BigDecimal total = vista.precioActual()
279 |         .multiply(BigDecimal.valueOf(cantidad)).setScale(2);
280 |     agregar.setText("Añadir al carrito    Q" + total.toPlainString());
281 | }
282 |
283 | private void confirmar() {
284 |     try {
285 |         VistaPresentacion vista = vistaActual();
286 |         if (vista == null) {
287 |             throw new IllegalArgumentException("Selecciona una presentación.");
288 |         }
289 |         List<OpcionPedido> opciones = vista.crearOpciones();
290 |         LineaPedido linea = new LineaPedido(UUID.randomUUID().toString(),
291 |             producto.idProducto(), vista.presentacion.idPresentacion(),
292 |             producto.nombre(), vista.presentacion.nombre(),
293 |             vista.presentacion.precio(), opciones, cantidad);
294 |         error.setText(" ");
295 |         alAgregar.accept(linea);
296 |     } catch (IllegalArgumentException excepcion) {
297 |         error.setText("<html>" + escaparHtml(excepcion.getMessage()) + "</html>");
298 |     }
299 |
300 | private void cambiar(int incremento) {
301 |     if (bloqueado() || tabla.getSelectedRow() < 0) return;
302 |     LineaPedido l = new ArrayList<>(lineas.values()).get(tabla.getSelectedRow());
303 |     int n = l.cantidad() + incremento;
304 |     if (n <= 0) lineas.remove(l.idLinea());
305 |     else if (n <= 999) lineas.put(l.idLinea(), l.conCantidad(n));
306 |     actualizar();
307 | }
308 |
309 | private boolean bloqueado() { return ocupado || pendiente != null || recuperacionFallida; }
310 | private BigDecimal suma() { return lineas.values().stream().map(LineaPedido::subtotal).reduce(new BigDecimal("0...
311 | private void actualizar() {
312 |     int seleccion = tabla.getSelectedRow(); modelo.setRowCount(0);
313 |     for (LineaPedido l : lineas.values()) modelo.addRow(new Object[]{l.resumen(), l.cantidad(), "Q" + l.subtota...
314 |     if (seleccion >= 0 && seleccion < modelo.getRowCount()) tabla.setRowSelectionInterval(seleccion, seleccion);
315 |     total.setText("Total: Q" + suma().toPlainString());
316 |     contador.setText(modelo.getRowCount() + (modelo.getRowCount() == 1 ? " producto" : " productos"));
317 |     cobrar.setText(pendiente == null ? "Continuar al pago" : "Reintentar cobro pendiente");
318 |     cobrar.setEnabled(!ocupado && !recuperacionFallida && !lineas.isEmpty());
319 | }
```

| Línea o concepto     | Qué significa y por qué importa                                                          |
|----------------------|------------------------------------------------------------------------------------------|
| actualizarPrecio     | Multiplica el precio actual de la presentación por la cantidad y actualiza el botón.     |
| crearOpciones        | Convierte los controles seleccionados en OpcionPedido/ProductoPedido/ModificacionPedido. |
| alAgregar.accept     | Callback: entrega la LineaPedido terminada al panel del carrito.                         |
| Map idLinea -> línea | Conserva configuraciones separadas; put agrega o reemplaza por ID.                       |
| cambiar              | Crea una copia con nueva cantidad o elimina si llega a cero.                             |
| suma                 | stream + map + reduce suma todos los subtotales con BigDecimal.                          |
| actualizar           | Reconstruye filas, total, contador y estado de botones.                                  |

## 16. Cobro: la transacción más importante

### Unidad atómica de cobro



| Concepto             | Por qué es necesario                                                                      |
|----------------------|-------------------------------------------------------------------------------------------|
| setAutoCommit(false) | Nada se confirma automáticamente; el DAO controla la unidad completa.                     |
| FOR UPDATE           | Bloquea la fila durante la transacción para que otra caja no use el mismo stock.          |
| commit               | Confirma pedido, detalle, elecciones, modificaciones y salidas juntos.                    |
| rollback             | Si una parte falla, revierte todo y evita pedidos sin inventario o inventario sin pedido. |
| idempotencia         | Reintentar la misma clave devuelve el comprobante previo y no cobra dos veces.            |

## 17. PagoDAO: líneas que deben saber defender

```

19 | public String cobrar(SolicitudPago pago) throws SQLException {
20 |     try (Connection c = conexiones.abrir()) {
21 |         if (c == null) throw new SQLException("No se pudo conectar con MySQL. Reintente cuando esté disponible.");
22 |         c.setAutoCommit(false);
23 |         boolean confirmando = false;
24 |         try {
25 |             // La clave se conserva al reintentar. El INSERT bloquea los intentos simultáneos.
26 |             ejecutar(c, "INSERT INTO pago_operacion(clave,solicitud,referencia) VALUES(?,?,?) "
27 |                 + "ON DUPLICATE KEY UPDATE clave=clave", pago.clave(), pago.toString(), pago.referencia());
28 |             try (PreparedStatement s = c.prepareStatement("SELECT solicitud,comprobante FROM pago_operacion WHE...
29 |                 s.setString(1, pago.clave());
30 |                 try (ResultSet r = s.executeQuery()) {
31 |                     r.next();
32 |                     if (!pago.toString().equals(r.getString(1))) throw new IllegalArgumentException("La operaci...
33 |                     String previo = r.getString(2);
34 |                     if (previo != null) {
35 |                         c.rollback();
36 |                     }
37 |                 }
38 |             }
39 |             Consumo consumo = new Consumo();
40 |             for (LineaPedido linea : pago.lineas()) validarLinea(c, linea, consumo);
41 |             bloquearInventario(c, consumo);
42 |             int id;
43 |             try (PreparedStatement s = c.prepareStatement("INSERT INTO pedido(numero_orden,id_usuario,tiposervicio,...
44 |                 s.setInt(1, pago.usuario()); s.setString(2, pago.servicio()); s.setString(3, pago.metodo());
45 |                 s.setBigDecimal(4, pago.recibido()); s.setBigDecimal(5, pago.total()); s.executeUpdate();
46 |                 try (ResultSet r = s.getGeneratedKeys()) { if (!r.next()) throw new SQLException("No se obtuvo ...
47 |             }
48 |             ejecutar(c, "UPDATE pedido SET numero_orden=? WHERE id_pedido=?", id, id);
49 |             for (LineaPedido linea : pago.lineas()) guardarLinea(c, id, linea);
50 |             descontarInventario(c, id, consumo);
51 |             String ticket = comprobante(id, pago);
52 |             ejecutar(c, "UPDATE pago_operacion SET id_pedido=?,comprobante=? WHERE clave=?", id, ticket, pago.c...
53 |             confirmando = true;
54 |             c.commit();
55 |             return ticket;
56 |         } catch (SQLException | RuntimeException error) {
57 |             try { c.rollback(); } catch (SQLException rollback) { error.addSuppressed(rollback); }
58 |             if (confirmando) throw new SQLException("No se pudo verificar la confirmación. Reintente este mismo...
59 |         }
60 |     }
61 | }

```

| Línea o concepto | Qué significa y por qué importa                                                 |
|------------------|---------------------------------------------------------------------------------|
| 20-22            | Abre conexión e inicia la transacción.                                          |
| 26-27            | INSERT ... ON DUPLICATE KEY crea o reconoce la misma operación.                 |
| 28               | Bloquea pago_operacion por clave.                                               |
| 32-40            | Si la clave pertenece a otro pedido, falla; si ya hay comprobante, lo devuelve. |
| 44-49            | Vuelve a verificar que la sesión siga activa y autorizada.                      |
| 51-53            | Calcula consumo de productos/ingredientes y bloquea inventario.                 |
| 55-64            | Guarda cabecera, número, árbol del pedido, inventario y comprobante.            |
| 66               | commit confirma todos los cambios.                                              |
| 68-71            | rollback revierte si ocurre SQLException o una regla inválida.                  |

## 18. Inventario, recetas y movimientos

```

226 | private void bloquearInventario(Connection c, Consumo consumo) throws SQLException {
227 |     for (Map.Entry<Integer,BigDecimal> entrada : consumo.productos.entrySet()) {
228 |         try (PreparedStatement s = c.prepareStatement("SELECT nombre,stock_actual,estado FROM producto WHERE id...
229 |             s.setInt(1, entrada.getKey());
230 |             try (ResultSet r = s.executeQuery()) {
231 |                 if (!r.next() || !r.getBoolean("estado")
232 |                     || BigDecimal.valueOf(r.getInt("stock_actual")).compareTo(entrada.getValue()) < 0)
233 |                     throw new IllegalArgumentException("No hay existencias suficientes de un producto del pedid...
234 |             }
235 |         }
236 |     }
237 |     for (Map.Entry<Integer,BigDecimal> entrada : consumo.ingredientes.entrySet()) {
238 |         try (PreparedStatement s = c.prepareStatement("SELECT nombre,stock_actual,estado FROM ingrediente WHERE...
239 |             s.setInt(1, entrada.getKey());
240 |             try (ResultSet r = s.executeQuery()) {
241 |                 if (!r.next() || !r.getBoolean("estado")
242 |                     || r.getBigDecimal("stock_actual").compareTo(entrada.getValue()) < 0)
243 |                     throw new IllegalArgumentException("No hay ingredientes suficientes para preparar el pedido...
244 |             }
245 |         }
246 |     }
247 | }

265 | ... private void descontarInventario(Connection c, int idPedido, Consumo consumo) throws SQLException {
266 |     for (Map.Entry<Integer,BigDecimal> entrada : consumo.productos.entrySet()) {
267 |         ejecutar(c,"UPDATE producto SET stock_actual=stock_actual-? WHERE id_producto=?",entrada.getValue().int...
268 |         ejecutar(c,"INSERT INTO movimiento_inventario(id_producto,id_pedido,tipo_movimiento,cantidad,motivo) VA...
269 |     }
270 |     for (Map.Entry<Integer,BigDecimal> entrada : consumo.ingredientes.entrySet()) {
271 |         ejecutar(c,"UPDATE ingrediente SET stock_actual=stock_actual-? WHERE id_ingrediente=?",entrada.getValue...
272 |         ejecutar(c,"INSERT INTO movimiento_inventario(id_ingrediente,id_pedido,tipo_movimiento,cantidad,motivo)...
273 |     }

```

| Línea o concepto      | Qué significa y por qué importa                                                     |
|-----------------------|-------------------------------------------------------------------------------------|
| productos             | Para tipo_stock=DIRECTO se comprueba y descuenta producto.stock_actual.             |
| ingredientes          | Para tipo_stock=RECETA se comprueba y descuenta ingrediente.stock_actual.           |
| compareTo < 0         | Detecta que lo disponible es menor que lo requerido.                                |
| guardarLinea          | Expande línea -> opciones -> productos -> modificaciones mediante claves generadas. |
| movimiento_inventario | Cada descuento genera una SALIDA ligada al id_pedido para auditoría.                |

Una modificación SIN no consume la cantidad\_default de ese ingrediente. Un EXTRA agrega cantidad\_extra por el número elegido. El backend vuelve a validar permite\_quitar, permite\_extra y max\_extras antes de cobrar.

# 19. Navegación y módulos del administrador

```
486 | private void configurarSecciones() {
487 |     navegador = new CardLayout();
488 |     panelContenido.removeAll();
489 |     panelContenido.setLayout(navegador);
490 |     agregarSeccion(new DashboardPanel(), DASHBOARD);
491 |     agregarSeccion(new UsuariosPanel(), USUARIOS);
492 |     agregarSeccion(new GestionMenuPanel(), GESTION_MENU);
493 |     agregarSeccion(new InventarioPanel(), INVENTARIO);
494 |     agregarSeccion(new ReportesPanel(), REPORTES);
495 | }
496 |
497 | private void agregarSeccion(JPanel seccion, String nombre) {
498 |     seccion.setBackground(new Color(248, 249, 251));
499 |     JPanel contenedor = new JPanel(new GridBagLayout());
500 |     contenedor.setBackground(new Color(248, 249, 251));
501 |     contenedor.add(seccion);
502 |     panelContenido.add(contenedor, nombre);
503 | }
504 |
505 | private void mostrarSeccion(String nombre, BotonMenuLateral botonSeleccionado) {
506 |     if (botonActivo != null && botonActivo != botonSeleccionado) {
507 |         botonActivo.setSeleccionado(false);
508 |     }
509 |     botonSeleccionado.setSeleccionado(true);
510 |     botonActivo = botonSeleccionado;
511 |     navegador.show(panelContenido, nombre);
512 |     panelContenido.revalidate();
513 |     panelContenido.repaint();
514 |     if (barraTitulo != null) {
515 |         barraTitulo.setSeccion(nombreSeccion(nombre));
516 |     }
517 | }
```

| Línea o concepto       |                                                             | Qué significa y por qué importa                                        |
|------------------------|-------------------------------------------------------------|------------------------------------------------------------------------|
| CardLayout             |                                                             | Mantiene paneles dentro de una ventana y cambia el visible por nombre. |
| configurarSecciones    |                                                             | Crea Dashboard, Usuarios, Gestión de menú, Inventario y Reportes.      |
| mostrarSeccion         |                                                             | Desmarca el botón anterior, marca el actual y muestra el panel.        |
| revalidate/repaint     |                                                             | Recalcula el diseño y vuelve a dibujar después del cambio.             |
| barraTitulo.setSeccion |                                                             | Actualiza el texto superior con la ubicación actual.                   |
| Módulo                 | Qué administra                                              | Clases centrales                                                       |
| Usuarios               | Roles, estado, turno, horario y contraseña.                 | UsuariosPanel, UsuarioFormPanel, UsuarioCRUD                           |
| Menú                   | Productos, precios, horario, stock, imagen y configuración. | GestionMenuPanel, ProductoCRUD, ConfiguracionMenuDAO                   |
| Inventario             | Existencias, ingredientes y movimientos.                    | InventarioPanel, IngredientesPanel, InventarioDAO                      |
| Reportes               | Ventas, pedidos, ticket, métodos y cancelaciones.           | ReportesPanel, ReporteDAO                                              |

## 20. DAO, CRUD y consultas JDBC

```
String sql = "SELECT nombre FROM producto WHERE id_producto=?";
try (Connection c = conectar();
    PreparedStatement ps = c.prepareStatement(sql)) {
    ps.setInt(1, idProducto);
    try (ResultSet rs = ps.executeQuery()) {
        if (rs.next()) nombre = rs.getString("nombre");
    }
}
```

| Pieza                          | Función                                                   |
|--------------------------------|-----------------------------------------------------------|
| Connection                     | Sesión abierta con MySQL.                                 |
| PreparedStatement              | SQL precompilado con ?; separa instrucción y datos.       |
| setInt/setString/setBigDecimal | Asigna los parámetros en orden empezando en 1.            |
| executeQuery                   | Ejecuta SELECT y devuelve ResultSet.                      |
| executeUpdate                  | Ejecuta INSERT/UPDATE/DELETE y devuelve filas afectadas.  |
| ResultSet.next                 | Avanza a una fila; false significa fin o resultado vacío. |
| getGeneratedKeys               | Obtiene el AUTO_INCREMENT creado por un INSERT.           |

CRUD describe operaciones básicas de un catálogo. DAO representa acceso a datos orientado a un caso de uso más amplio, como cobrar, cargar el árbol de menú o calcular reportes. En este proyecto ambos conceptos conviven.

## 21. SQL que deben saber leer

| Elemento SQL | Pregunta que responde                             | Ejemplo en el proyecto            |
|--------------|---------------------------------------------------|-----------------------------------|
| SELECT       | ¿Qué columnas quiero consultar?                   | producto, pedidos, reportes       |
| FROM         | ¿De qué tabla parte la consulta?                  | FROM producto p                   |
| JOIN ... ON  | ¿Cómo relaciono tablas?                           | receta con ingrediente            |
| WHERE        | ¿Qué filas cumplen condiciones?                   | estado=TRUE, categoría=?          |
| EXISTS       | ¿Existe al menos una fila relacionada?            | producto con presentación activa  |
| COALESCE     | ¿Qué valor alternativo uso si el primero es NULL? | precio presentación o precio_base |
| GROUP BY     | ¿Cómo agrupo para contar/sumar?                   | ventas por método/cajero          |
| ORDER BY     | ¿En qué orden muestro?                            | predeterminada primero            |
| LIMIT        | ¿Cuántas filas necesito?                          | una presentación preferida        |
| INSERT       | ¿Cómo creo una fila?                              | pedido y sus niveles              |
| UPDATE       | ¿Cómo cambio una fila?                            | stock_actual=stock_actual-?       |
| FOR UPDATE   | ¿Cómo bloqueo la fila durante la transacción?     | stock y pago_operacion            |

```
SELECT p.nombre, c.nombre AS categoria
FROM producto p
JOIN categoria c ON c.id_categoria = p.id_categoria
WHERE p.estado = TRUE
ORDER BY c.nombre, p.nombre;
```

Los alias p y c son nombres cortos para tablas. AS categoria cambia el nombre de la columna en el resultado. JOIN no copia datos: combina filas relacionadas para esa consulta.

## 22. Reportes, paginación y trabajo en segundo plano

| Concepto         | Implementación                                    | Razón                                                             |
|------------------|---------------------------------------------------|-------------------------------------------------------------------|
| Resumen diario   | ReporteDAO.cargar(LocalDate)                      | Calcula ventas, pedidos, ticket y métodos del rango del día.      |
| Límites de fecha | inicio inclusivo y día siguiente exclusivo        | Incluye todas las horas sin depender de 23:59:59.                 |
| Filtros          | cajero, método, servicio y búsqueda               | Permite explorar el mismo resultado sin mezclar la lógica visual. |
| Paginación       | paginaActual + tamaño de página                   | No dibuja cientos de filas simultáneamente.                       |
| SwingWorker      | consulta en doInBackground, actualización en done | Evita congelar el EDT.                                            |
| CSV              | ReporteCsv / exportadores                         | Formato simple que abre Excel sin librerías pesadas.              |
| Renderizadores   | TableCellRenderer                                 | Cambian color/formato sin modificar el dato real.                 |

```
LocalDateTime inicio = fecha.atStartOfDay();
LocalDateTime fin = fecha.plusDays(1).atStartOfDay();
// SQL: fecha hora >= inicio AND fecha hora < fin
```

Pregunta frecuente: ¿por qué no usar DATE(fecha\_hora)=? Siempre? El rango puede aprovechar mejor un índice sobre fecha\_hora y define con claridad los límites.

## 23. Validaciones, errores y riesgos que deben reconocer

| Riesgo              | Protección actual                                   | Mejora futura                                              |
|---------------------|-----------------------------------------------------|------------------------------------------------------------|
| SQL injection       | PreparedStatement con parámetros.                   | No concatenar texto del usuario.                           |
| Contraseña expuesta | Hash PBKDF2-SHA256 y migración de cuentas antiguas. | Nunca volver a texto plano; considerar BCrypt/Argon2.      |
| Credenciales DB     | Conexión centralizada.                              | Mover secretos fuera del código.                           |
| Cobro duplicado     | pago_operacion y clave idempotente.                 | Persistir/monitorizar reintentos de caja.                  |
| Venta sin stock     | FOR UPDATE y revalidación en PagoDAO.               | Pruebas concurrentes con varias cajas.                     |
| Datos a medias      | Transacción commit/rollback.                        | Mantener toda nueva escritura del cobro en la transacción. |
| Ventana congelada   | SwingWorker en cargas administrativas.              | No ejecutar consultas lentas en listeners del EDT.         |
| Form corrupto       | Bloques protegidos de NetBeans.                     | Editar initComponents desde GUI Builder.                   |
| Borrado de historia | estado como borrado lógico.                         | No eliminar filas usadas por pedidos.                      |

Diferencia importante: validar en la interfaz mejora la experiencia, pero validar en el backend/DAO protege la integridad. El cajero puede mostrar una regla antigua; PagoDAO debe volver a leer precio, grupo, opción, receta y stock antes de confirmar.

## 24. Pruebas y diagnóstico

| Prueba                        | Qué cubre                                                                |
|-------------------------------|--------------------------------------------------------------------------|
| PagosTest                     | Validaciones de pago, reintento e inconsistencias.                       |
| ReporteDAOTest / ReportesTest | Fechas, indicadores, CSV y resultados vacíos.                            |
| InventarioPanelTest           | Navegación, datos y tabla sin imágenes.                                  |
| MenuAnimacionTest             | Apertura/cierre y textos del menú.                                       |
| FormReportesTest              | Integridad del formulario de reportes.                                   |
| ValidarDatosMenu              | Ejecuta scripts dos veces, busca duplicados y configuraciones inválidas. |
| Síntoma                       | Ruta de revisión                                                         |
| No conecta                    | Servicio MySQL → puerto/base → credenciales → driver JDBC.               |
| Producto no aparece           | estado → categoría → presentación activa → horario → subcategoría.       |
| No aparece WIMenú             | presentacion MENU → grupos → opciones → componentes activos.             |
| No deja agregar               | mínimo/máximo → opción elegida → componente → personalización.           |
| Falla al cobrar               | sesión → precio/reglas → stock → transacción → mensaje raíz.             |
| Stock incorrecto              | tipo_stock → receta → SIN/EXTRA → movimiento ligado al pedido.           |
| Formulario solo lectura       | marcas GEN-BEGIN/GEN-END y sincronía .java/.form.                        |

## 25. Guion de demostración en 10 minutos

| Tiempo     | Qué mostrar    | Qué decir                                                  |
|------------|----------------|------------------------------------------------------------|
| 0:00-0:45  | Login          | Autenticación parametrizada, hash, estado y rol.           |
| 0:45-1:30  | Arquitectura   | Vista -> modelos -> DAO/CRUD -> MySQL.                     |
| 1:30-3:30  | Big Mac        | Individual/WIMenú, grupos, opción con recargo y SIN/EXTRA. |
| 3:30-4:30  | Combo          | Dos productos internos pueden tener cambios distintos.     |
| 4:30-6:00  | Carrito y pago | BigDecimal, SolicitudPago, transacción e idempotencia.     |
| 6:00-7:00  | Inventario     | RECETA vs DIRECTO y movimientos automáticos.               |
| 7:00-8:30  | Administrador  | Usuarios, menú configurable, inventario y reportes.        |
| 8:30-9:30  | Base de datos  | Jerarquía de menú y jerarquía histórica del pedido.        |
| 9:30-10:00 | Cierre         | Consistencia, pruebas y mejoras futuras.                   |

**Consejo:** una persona maneja la aplicación y otra explica el flujo técnico. No lean código completo en pantalla; señalen método, condición, consulta y efecto. Si ocurre un error durante la demo, expliquen la ruta de diagnóstico en vez de ocultarlo.

Frase de cierre sugerida: 'El diseño separa la configuración comercial de los productos físicos, por eso el mismo modelo soporta individuales, menús, Cajitas y combos, conserva personalizaciones por componente y mantiene el inventario consistente al cobrar.'

## 26. Preguntas técnicas probables – parte 1

### ¿Por qué un combo no es un producto simple?

Porque cocina, precio e inventario necesitan conocer cada componente y cada modificación. El producto principal abre una presentación; sus opciones entregan productos internos.

### ¿Cómo una hamburguesa va sin pepinillo y la otra no?

Cada ProductoPedido tiene idInterno y su propia lista de ModificacionPedido. No se guarda el cambio en la línea completa.

### ¿Qué diferencia hay entre producto y presentacion\_menu?

Producto describe el artículo reutilizable y su receta. presentacion\_menu describe una forma de venderlo, su tipo y precio.

### ¿Para qué sirven minimo y maximo?

Obligan y limitan cuántas opciones pueden elegirse dentro de un grupo.

### ¿Por qué se guardan nombres repetidos en pedido\_\*?

Son snapshots históricos. Un cambio posterior del catálogo no altera el contenido de una venta ya realizada.

### ¿Por qué usan BigDecimal?

El dinero necesita decimales exactos. double representa valores binarios aproximados y puede introducir centavos incorrectos.

### ¿Qué evita una inyección SQL?

PreparedStatement separa la sentencia de los valores asignados con setString/setInt; no se concatena la entrada del usuario.

### ¿Qué es una transacción?

Una unidad de cambios que se confirma completa con commit o se deshace completa con rollback.

### ¿Qué hace FOR UPDATE?

Bloquea las filas leídas hasta terminar la transacción, evitando que dos cajas consuman simultáneamente el mismo stock.

### ¿Qué es idempotencia?

Repetir el mismo cobro lógico con la misma clave no crea una segunda venta; devuelve el resultado previo.

## 27. Preguntas técnicas probables – parte 2 y repaso final

### ¿Qué diferencia hay entre DAO y CRUD?

CRUD reúne operaciones básicas de un catálogo; DAO encapsula acceso a datos de un caso de uso más complejo, como cobrar o cargar el árbol del menú.

### ¿Por qué hay archivos .form y .java?

.form conserva el diseño editable de NetBeans. .java contiene comportamiento y el initComponents generado.

### ¿Por qué usar SwingWorker?

Para ejecutar consultas lentas fuera del EDT y evitar que la interfaz deje de responder.

### ¿Cómo funciona la disponibilidad por horario?

producto.disponibilidad\_menu guarda TODO\_DIA, DESAYUNO o ALMUERZO; HorarioMenu y los DAO filtran/rechazan fuera de horario.

### ¿Qué diferencia hay entre RECETA y DIRECTO?

RECETA descuenta ingredientes de producto\_ingrediente. DIRECTO descuenta unidades de producto.stock\_actual. NINGUNO no controla existencia directa.

### ¿Por qué no confiar solo en la interfaz?

La configuración puede cambiar antes del cobro. PagoDAO vuelve a validar reglas, precios, usuario e inventario contra la base.

### ¿Qué mejorarían?

Externalizar credenciales, añadir índices/pruebas concurrentes, usar migraciones versionadas, aumentar pruebas automatizadas y separar aún más servicios de la interfaz.

| Repaso en una línea | Respuesta                                                                 |
|---------------------|---------------------------------------------------------------------------|
| Arquitectura        | Vista -> Modelo -> DAO/CRUD -> MySQL.                                     |
| Menú                | Producto -> Presentación -> Grupo -> Opción -> Componente.                |
| Pedido              | Pedido -> Detalle -> Opción -> Producto -> Modificación.                  |
| Cobro               | Validar -> bloquear -> guardar -> descontar -> commit; error -> rollback. |
| Inventario          | DIRECTO descuenta producto; RECETA descuenta ingredientes.                |